Practical no. 18

Aim - Continuous Integration using Jenkins

Configure Jenkins to execute Selenium automation scripts automatically. Generate build reports and analyze test execution history.

Step 1: Install Jenkins

Download Jenkins - D:\STQA (file similar to jenkins.msi)

Double-click the downloaded .msi file and keep the default options.

Keep the Jenkins port as 8080.

Finish the installation. Jenkins will be available at: http://localhost:8080

Step 2: Start Jenkins

1. Press Win + R.

2. Type: services.msc

3. Press Enter and find Jenkins. Its status should say Running.

If it is not running, right-click Jenkins → Start.

Step 3: Open Jenkins

Open Chrome/Edge and enter: http://localhost:8080 (the "Unlock Jenkins" screen appears).

Step 4: Get the Initial Administrator Password

Open the following file using Notepad:

```
C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
```

Or open Command Prompt and run:

```
type C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
```

Copy the password and paste it into the Jenkins browser page.

Step 5: Complete Jenkins Setup

Choose: Install suggested plugins

Wait for the installation to finish.

Create your administrator account (Username: admin, Password: ********) and click Create.

Step 6: Reach the Jenkins Dashboard

Jenkins Dashboard opens at: http://localhost:8080

Step 7: Create Maven Project

Open Eclipse and select the workspace: C:\Users\ITCS\eclipse-workspace

File → New → Maven Project

Tick: Create a simple project → Next

Step 8: Enter Project Details

Group Id: com.example

Artifact Id: selenium-project

Leave the other settings as default → Finish

Step 9: Open pom.xml

In Project Explorer, double-click pom.xml and replace its contents with:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
   https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <groupId>com.selenium</groupId>
   <artifactId>selenium-project</artifactId>
   <version>1.0-SNAPSHOT</version>
   <properties>
      <maven.compiler.source>21</maven.compiler.source>
      <maven.compiler.target>21</maven.compiler.target>
   </properties>
   <dependencies>
      <dependency>
         <groupId>org.seleniumhq.selenium</groupId>
         <artifactId>selenium-java</artifactId>
         <version>4.35.0</version>
      </dependency>
      <dependency>
         <groupId>org.testng</groupId>
         <artifactId>testng</artifactId>
         <version>7.11.0</version>
         <scope>test</scope>
      </dependency>
   </dependencies>
</project>
```

Press Ctrl + S

Then: Right-click project → Maven → Update Project → OK

Wait until Maven downloads the dependencies.

Step 10: Create Package

Right-click src/test/java → New → Package

Enter: tests → Finish

Step 11: Create Java Class

Right-click tests → New → Class

Enter: LoginTest → Finish

Open LoginTest.java, delete everything and paste:

```
package tests;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
public class LoginTest {
   private WebDriver driver;
   @BeforeMethod
   public void setUp() {
      // Selenium Manager automatically finds/downloads the
      // appropriate ChromeDriver. No driver path or version required.
      driver = new ChromeDriver();
      driver.manage().window().maximize();
   }
   @Test
   public void loginTest() {
      driver.get("https://www.google.com");
      System.out.println("Title: " + driver.getTitle());
   }
   @AfterMethod
   public void tearDown() {
      if (driver != null) {
         driver.quit();
      }
   }
}
```

Save: Ctrl + S

Step 12: Create testng.xml

Right-click project selenium-project → New → File

Enter: testng.xml → Finish

Paste:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM
   "https://testng.org/testng-1.0.dtd">
<suite name="Selenium Test Suite">
   <test name="Chrome Tests">
      <classes>
         <class name="tests.LoginTest"/>
      </classes>
   </test>
</suite>
```

Save it.

Step 13: Run the Test

Right-click LoginTest.java → Run As → TestNG Test

If getting - An internal error occurred during: "Launching LoginTest". invalid version "1.8.0_461": non-numeric "0_461"

1. Window → Preferences → Java → Installed JREs → add/select your JDK 21.

2. Right-click selenium-project → Properties → Java Build Path → Libraries. If JRE System Library [JavaSE-1.8] is present, remove it and add JRE System Library → Workspace default JRE (or select JDK 21). Click Apply and Close.

3. Project → Properties → Java Compiler → Compiler compliance level: 21 → Apply and Close.

Step 14: Run Through Maven

Open Command Prompt and go to your Eclipse workspace/project:

```
cd eclipse-workspace\selenium-project
```

Run:

```
mvn clean test
```

If you get BUILD SUCCESS, you are ready for Jenkins.

If getting BUILD FAILURE then type the following commands:

```
C:\Users\ITCS>set JAVA_HOME=C:\Program Files\Java\jdk-21.0.12
C:\Users\ITCS >set PATH=%JAVA_HOME%\bin;%PATH%
C:\Users\ITCS>java -version
C:\Users\ITCS>javac -version
C:\Users\ITCS>mvn -version
C:\Users\ITCS>cd eclipse-workspace\selenium-project
C:\Users\ITCS\eclipse-workspace\selenium-project>mvn clean test
```

Step 15: Check the Generated Report

After running mvn clean test, Maven creates the target folder.

Inside target/surefire-reports you will find the XML test-result files.

Step 16: Refresh Eclipse

Right-click selenium-project → Refresh (or press F5), then expand target → surefire-reports.

Step 17: Open Jenkins

Open your browser: http://localhost:8080 and log in.

Step 18: Configure JDK and Maven

Manage Jenkins → Tools

JDK installation name: JDK21, JAVA_HOME: C:\Program Files\Java\jdk-21.0.12

Maven installation name: Maven, MAVEN_HOME: D:\STQA\apache-maven-3.9.16

Click Save.

Step 19: Create Jenkins Job

From the Jenkins Dashboard click New Item

Enter: Selenium-CI

Select: Freestyle project → OK

Step 20: Add the Build Command

Scroll down to Build Steps.

Click: Add build step → Execute Windows batch command

Enter:

```
@echo off
cd /d C:\Users\ITCS\eclipse-workspace\selenium-project
echo ========================================
echo Running Maven Test
echo ========================================
call mvn clean test
echo.
echo ========================================
echo Jenkins Maven Test Completed
echo ========================================
exit /b 0
```

Click Save, then Build Now.

Step 21: View Build Result

Check Build History and the console output of the Selenium-CI build.